



APRENDIZADO PROFUNDO

deep learning

PROF. JOSENALDE OLIVEIRA

PROGRAMA DE PÓS-GRADUAÇÃO EM TI (PPgTI) - UFRN

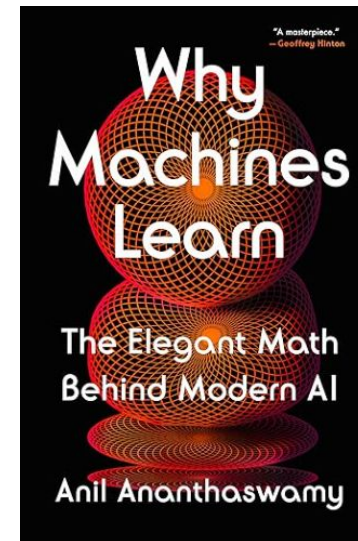
PERGUNTAS INQUIETANTES...

Teste de Turing (1950, tarefa linguística): a capacidade de uma máquina está à altura da inteligência humana?

Seria possível construir uma máquina que conseguisse ler (*natural language understanding*) e escrever (*natural language generation*)?

Premissa: o domínio da linguagem é sem sombra de dúvidas a maior capacidade cognitiva humana

- 30.11.2022 - Lançamento ChatGPT (GPT-3.5)
- A OpenAI foi criada em 12.2015
- 03.2023 - Lançamento Google BARD (02.2024-Gemini)

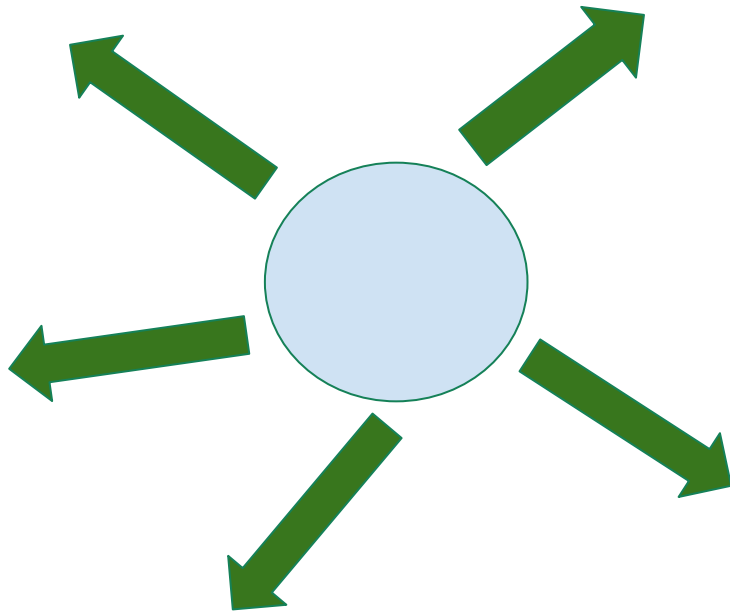


Alan Turing (1912-1954)

REDES RECORRENTES (e suas variantes) (*RNN*)

Cap. 10 Goodfellow et al.

Processamento/Modelagem (profundo) de sequências

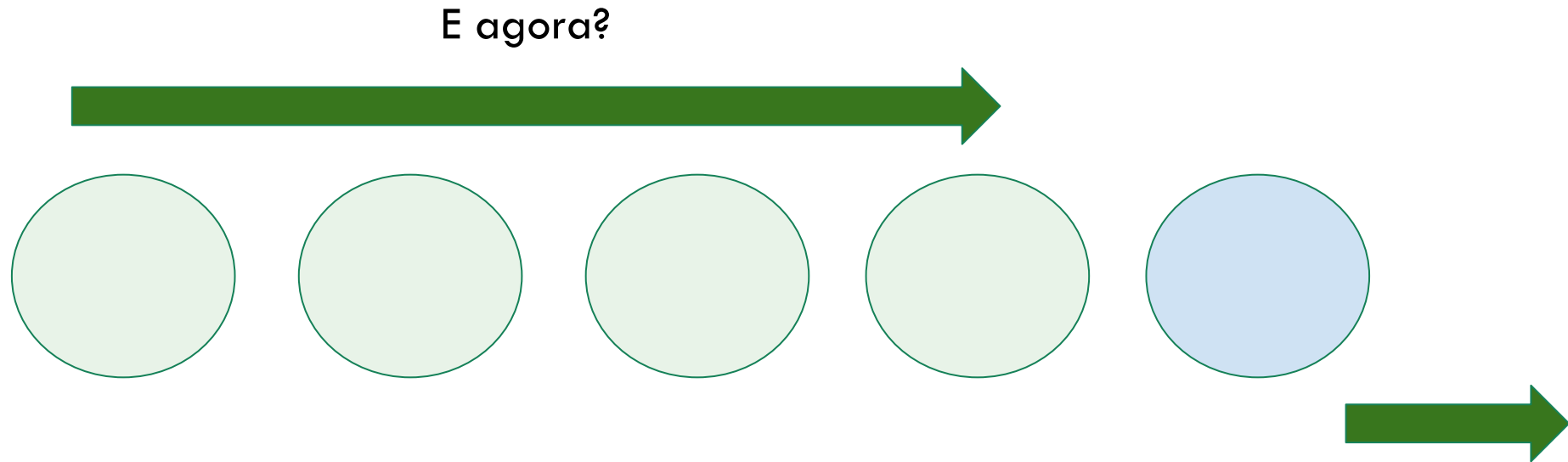


Dado este círculo no instante K.
Para onde ele vai no instante K+1?

Ideia adaptada de introtodeeplearning.com (MIT6.S191: youtube.com/watch?v=GvezxUdLrEk)

REDES RECORRENTES (e suas variantes) (*RNN*)

Processamento/Modelagem (profundo) de sequências



AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Texto/Linguagem Natural

PPGTI3003 - APRENDIZADO PROFUNDO T01 (2025.2)

Caracteres

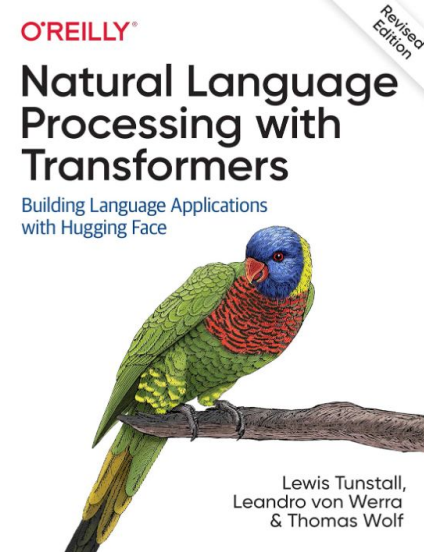
P
P
G
T
I
3
0
0
3

Palavras (sentenças)

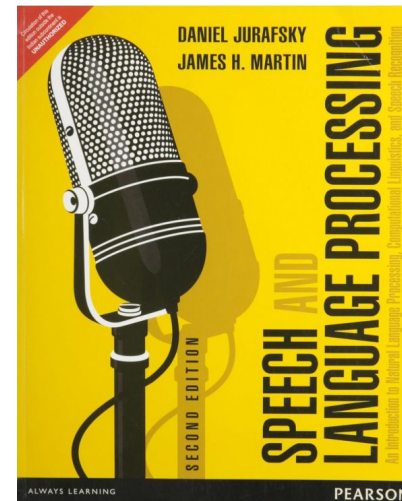
APRENDIZADO

+

PROFUNDO



2022



2014

AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Texto/Linguagem Natural (*position aware*)

Estive em Natal em 2019

Em 2019, estive em Natal.

Em que ano você esteve em Natal?

Você viajou entre 2010 e 2020?

Já conheceu cidades no Nordeste Brasileiro?

REQUISITOS:

- tamanho variável
- dependências de longo termo
- capturar contexto
- manter informação sobre ordem
- compartilhar parâmetros na sequência

France is where I grew up, but now I live in Boston. I speak fluent

_____.

AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Texto/Linguagem Natural

Estive em Natal em 2019

Em 2019, estive em Natal.

Em que ano você esteve em Natal?

Você viajou entre 2010 e 2020?

Já conheceu cidades no Nordeste Brasileiro?

Processar sequências longas

Processar sequências de tamanho variável

Compartilhar parâmetros (pesos) ao longo dos intervalos de tempo

UM PRIMEIRO EXPERIMENTO (2015) (gerar texto caracter a caracter - Char-RNN)



Andrej Karpathy

Modelo treinado em toda a obra de Shakespeare
Aprendeu palavras, gramática, pontuação, somente aprendendo a
predizer o próximo caractere - *modelo da distribuição de probabilidade
do próximo caractere de uma sequência levando em conta os caracteres
anteriores*

Pode ser usado para gerar UM NOVO TEXTO, um caractere de cada vez.

Modelo com RNN de três camadas, com 512 nós ocultos em cada camada

Ele fez também um pequeno experimento treinando o livro **Gerra e Paz** e
a cada N iterações exibia amostras do que estava sendo gerado:

UM PRIMEIRO EXPERIMENTO (gerar texto caracter a caracter - [Char-RNN](#))



[Andrej Karpathy](#)

#100

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgrd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

#300

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

#500

we counter. He stutn co des. His stanted out one ofler that concossions and was
to gearang reay Jotrets and with fre colt off paitt thin wall. Which das stimn

#700

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.

#1200

"Kite vouch!" he repeated by her
door. "But I would be done and quarts, feeling, then, son is people...."

UM PRIMEIRO EXPERIMENTO

(gerar texto caracter a caracter - [Char-RNN](#))



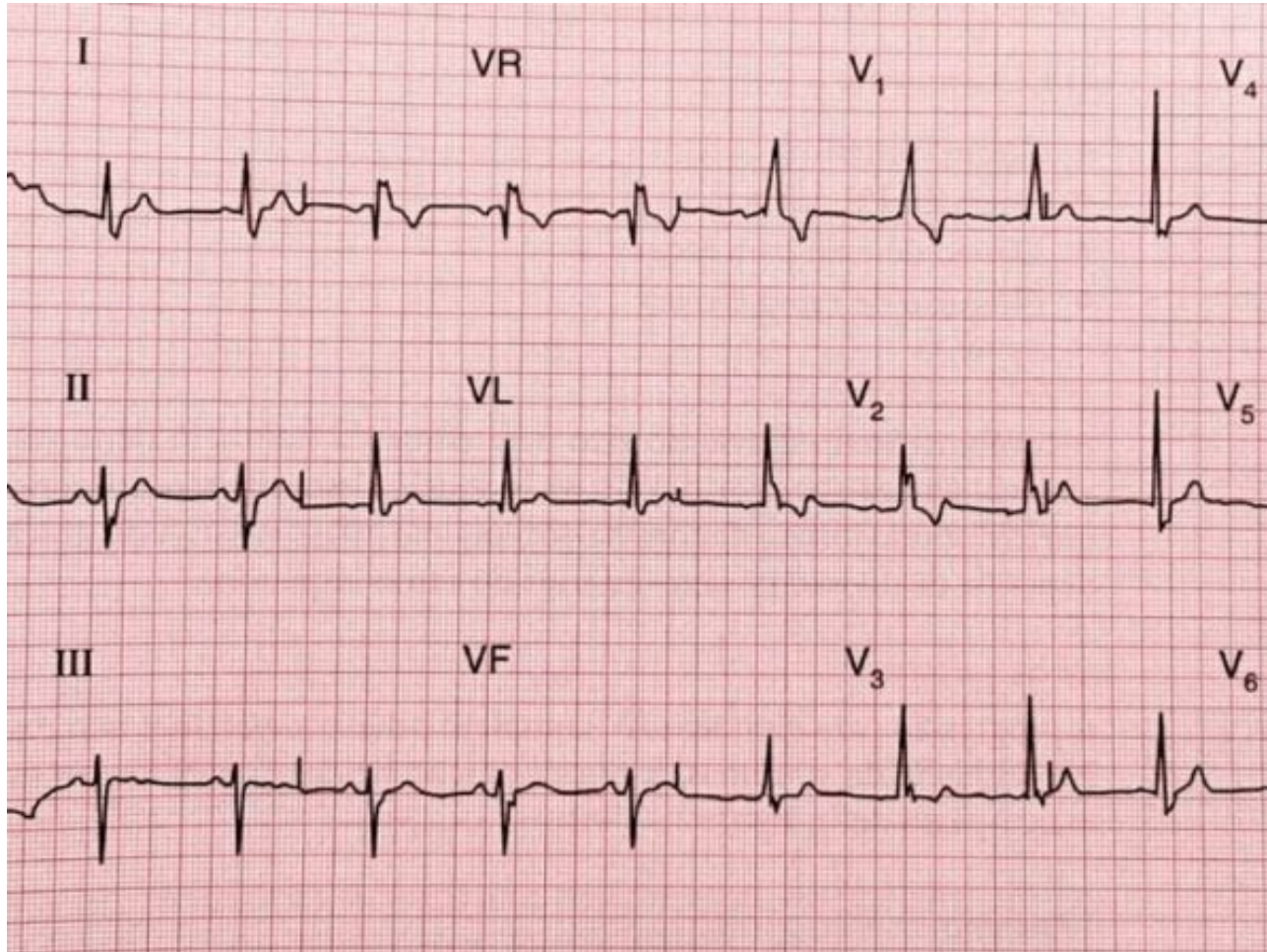
[Andrej Karpathy](#)

#2000

"Why do what that day," replied Natasha, and wishing to himself the fact the princess, Princess Mary was easier, fed in had oftended him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

AS SEQUÊNCIAS ESTÃO ENTRE NÓS

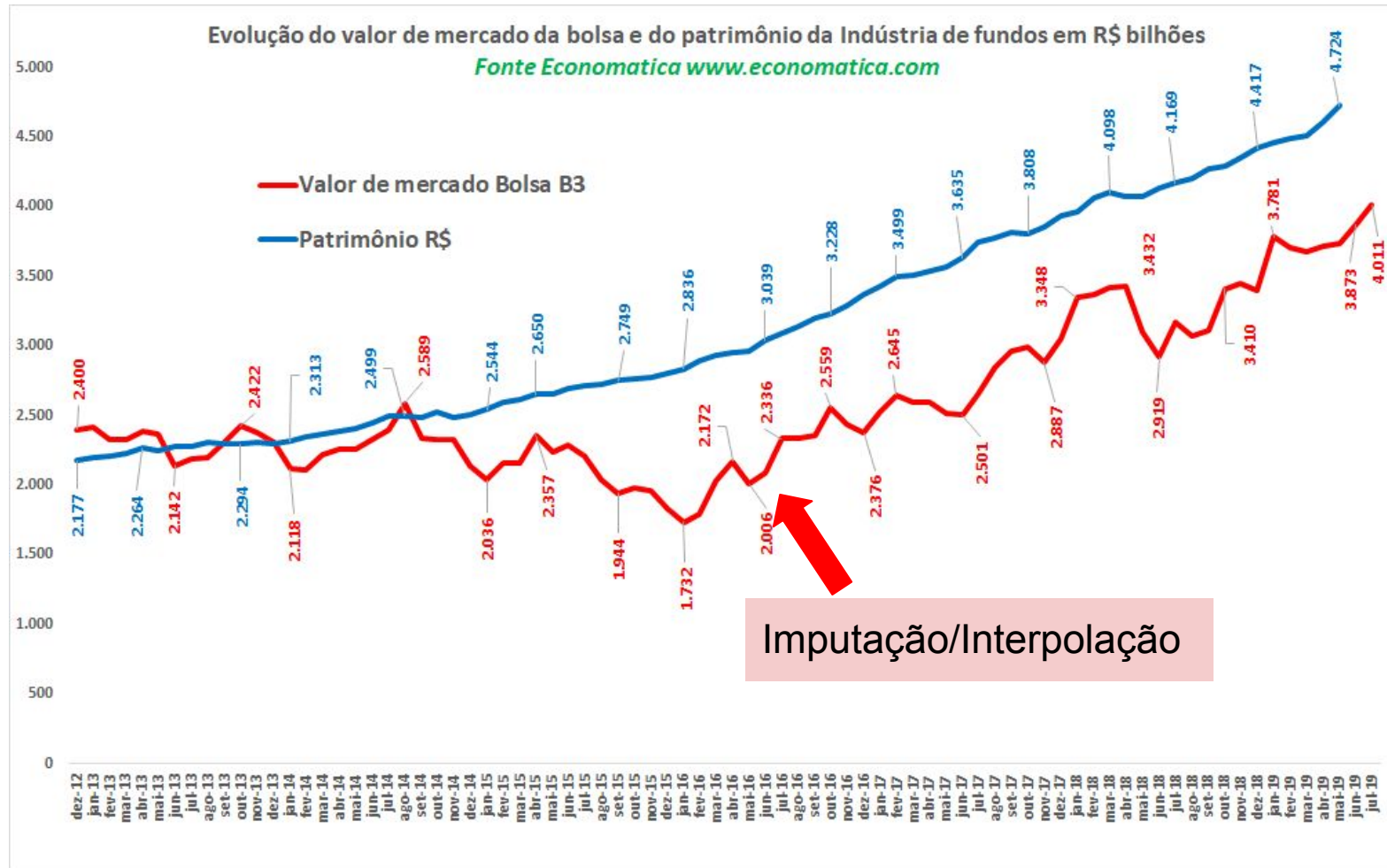
Sinais médicos: ECG (etc.)



AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Sistema financeiro: bolsa de valores, cotações (etc.) - séries temporais

[[Link](#)]



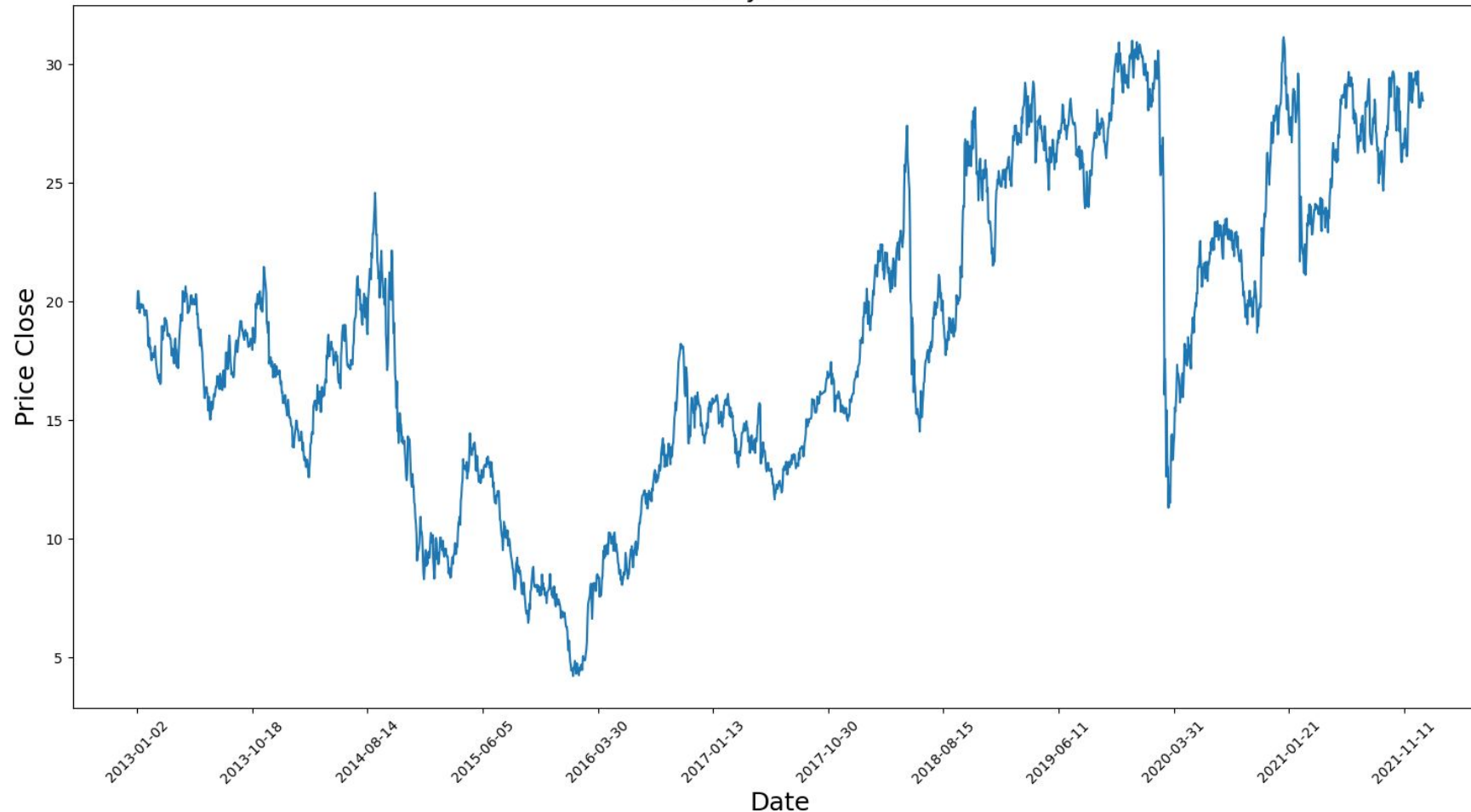
Extrapolação/Forecast

Imputação/Interpolação

AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Sistema financeiro: bolsa de valores, cotações (etc.) - séries temporais

History Petrobras

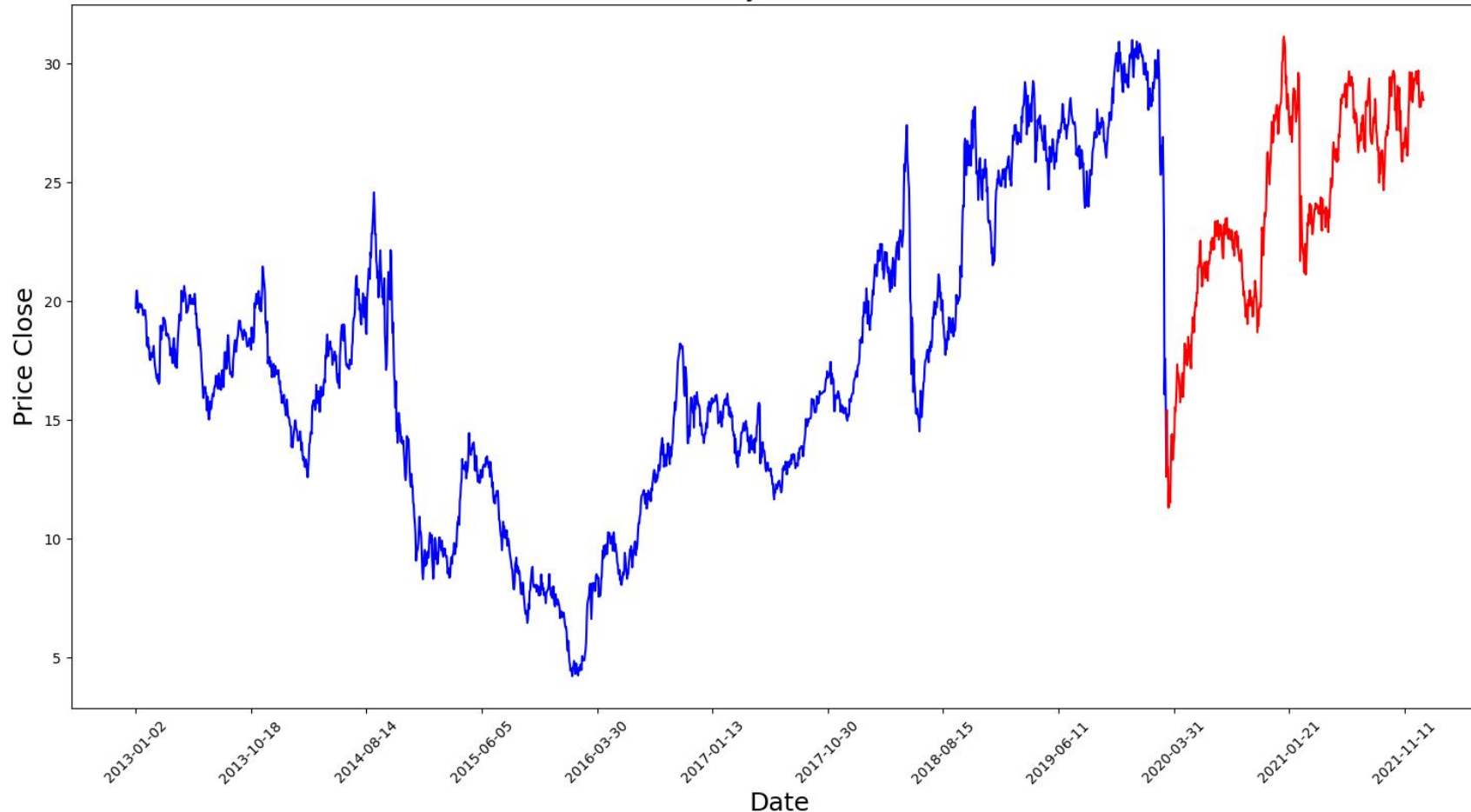


[[Link](#)]

AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Sistema financeiro: bolsa de valores, cotações (etc.) - séries temporais

History Petrobras

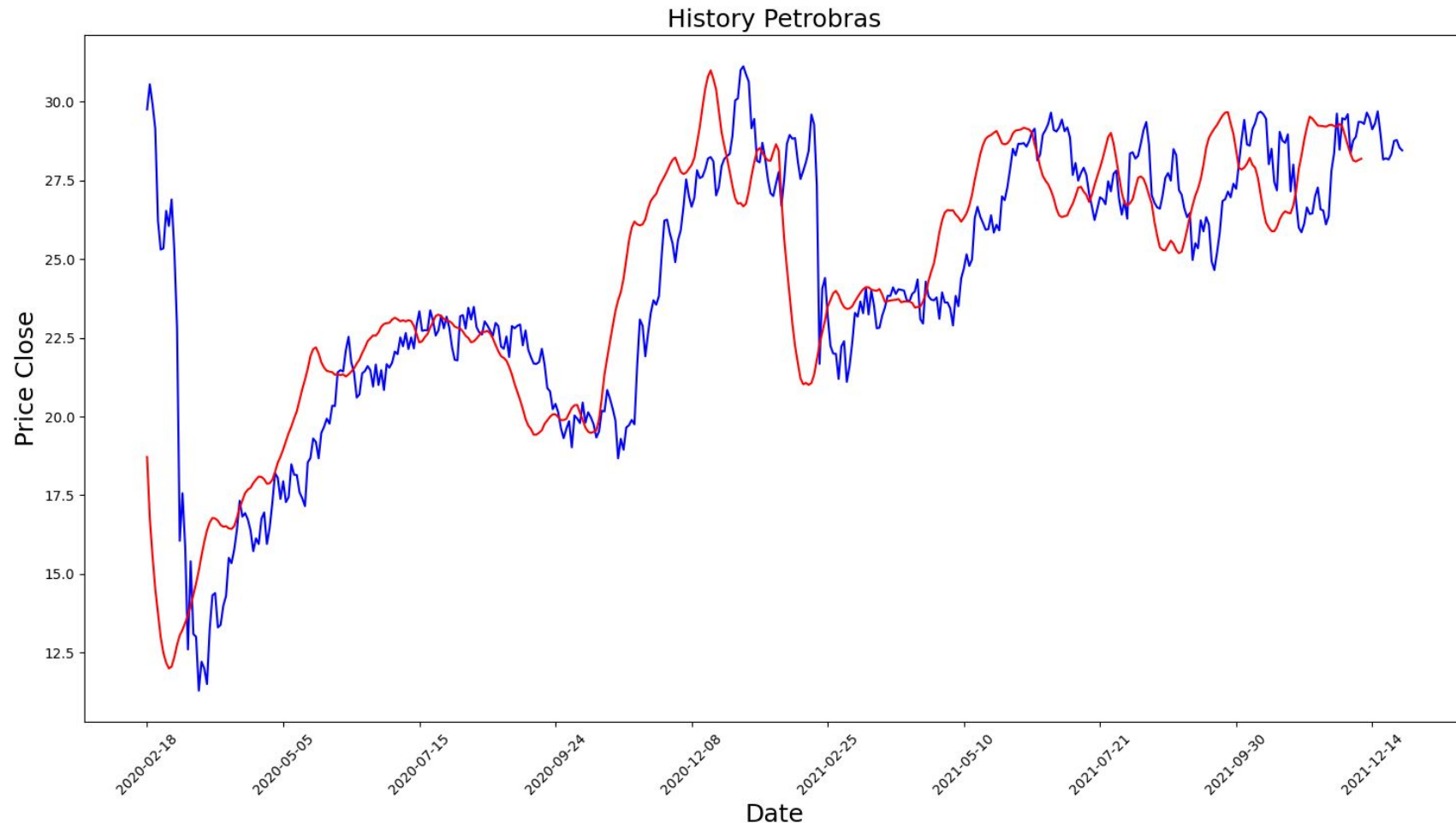


[[Link](#)]

AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Sistema financeiro: bolsa de valores, cotações (etc.) - séries temporais

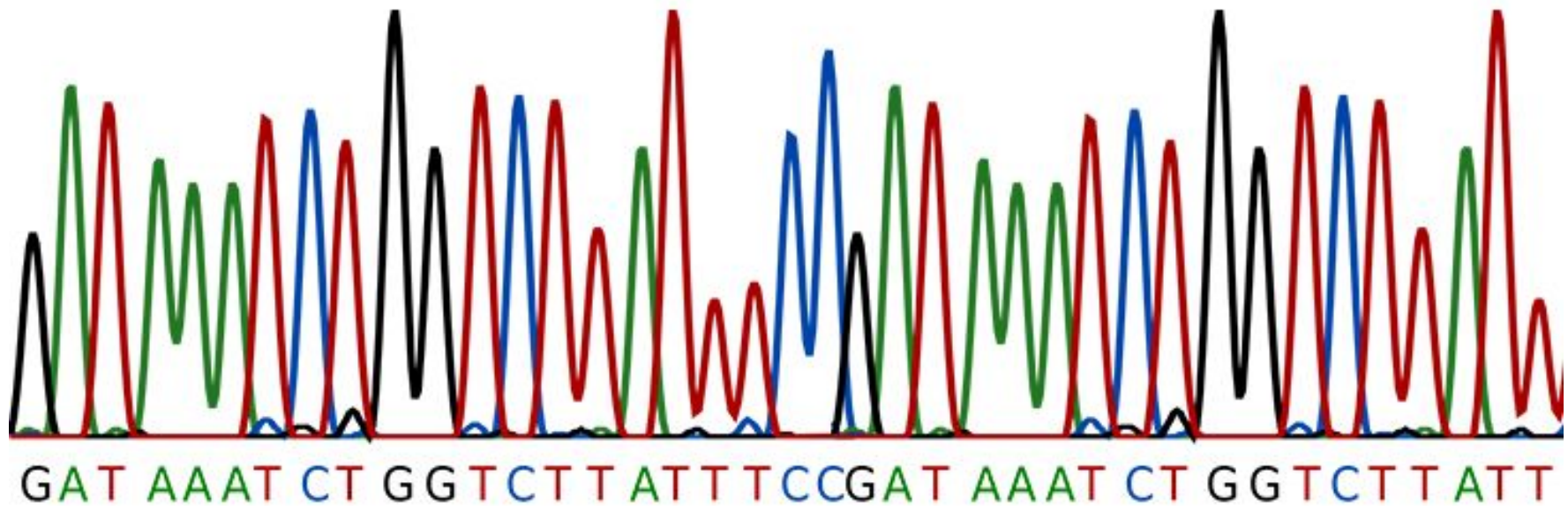
[\[Link\]](#)



AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Genética: sequenciamento DNA

SEQUENCIAMENTO DE DNA

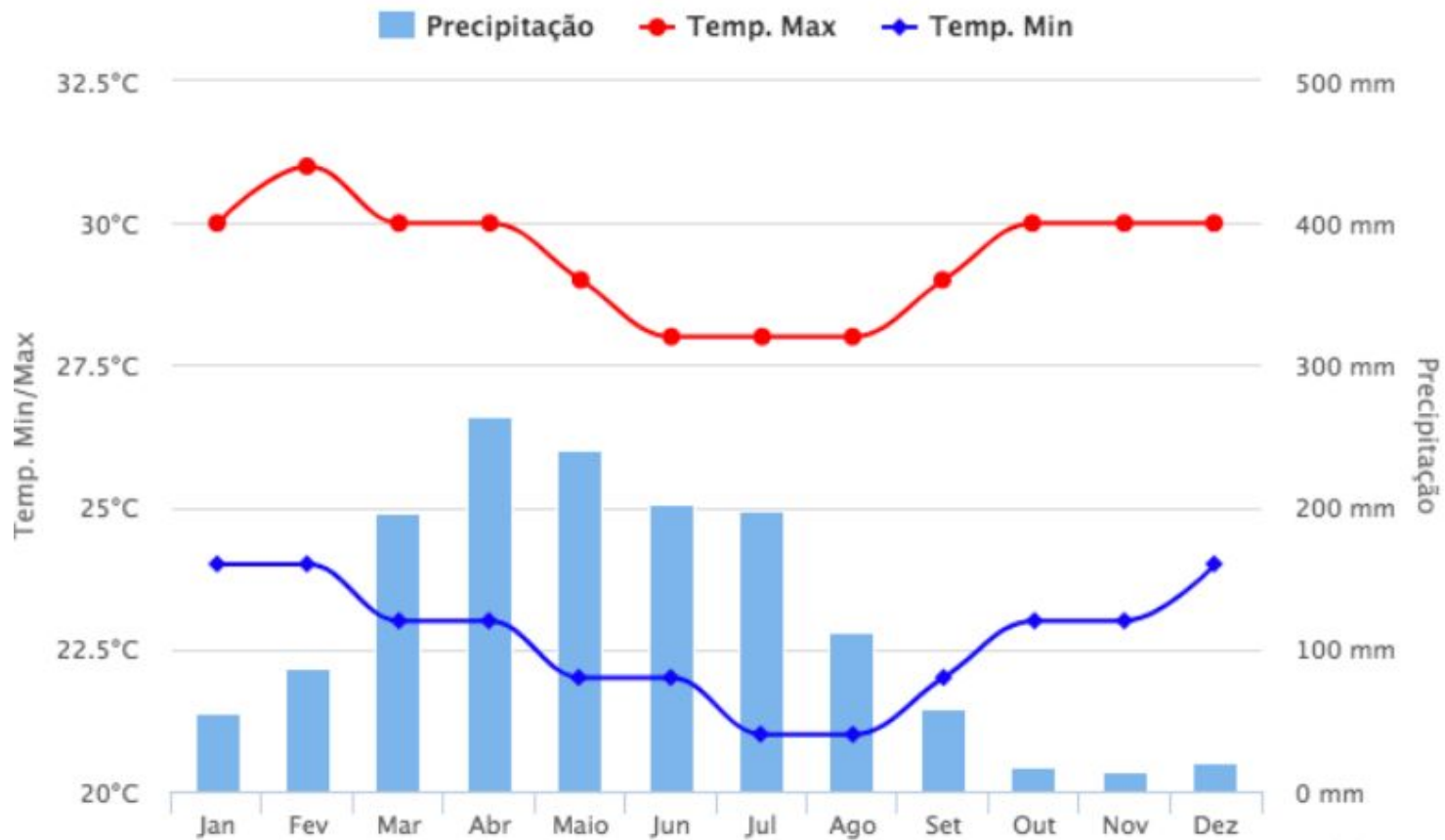


AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Meteorologia

Natal - RN

compartilhar



AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Áudio - tradução automática, reconhecimento de fala, transcrição audio2text



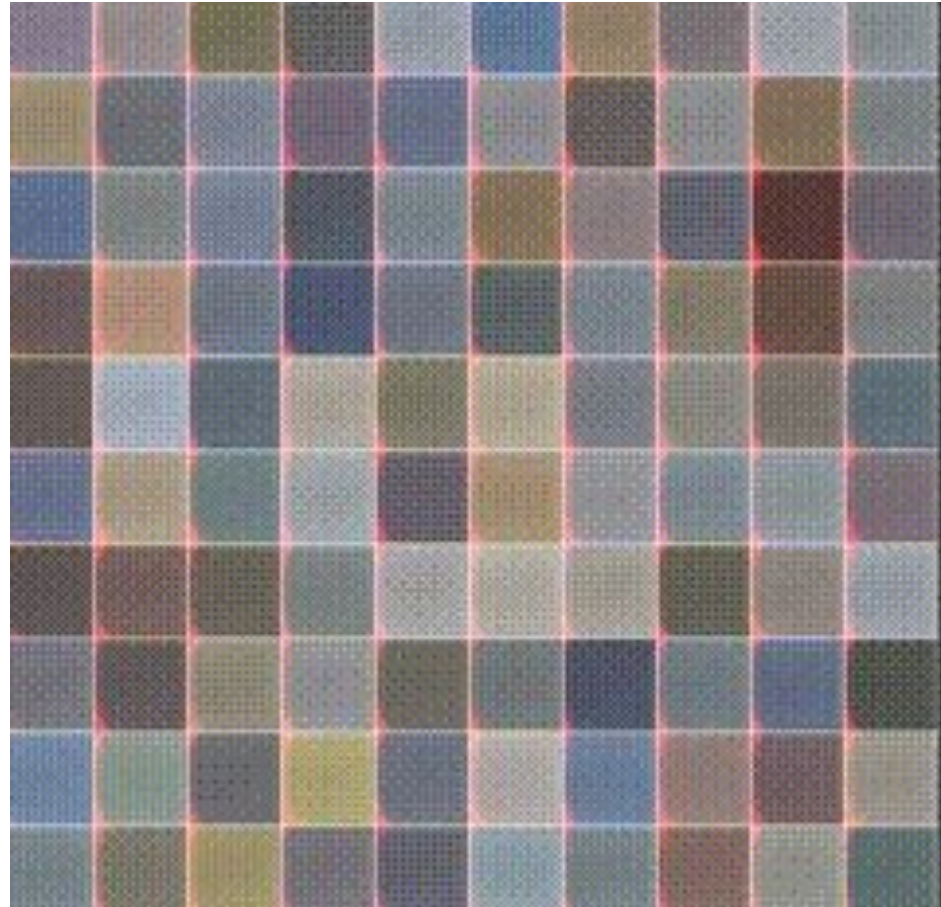
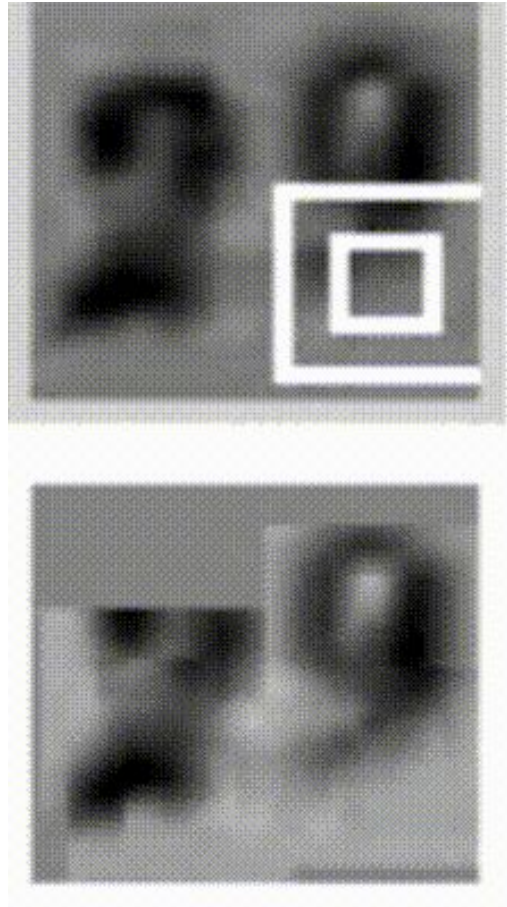
AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Vídeo (frames)



AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Imagens (sequências de pixels)



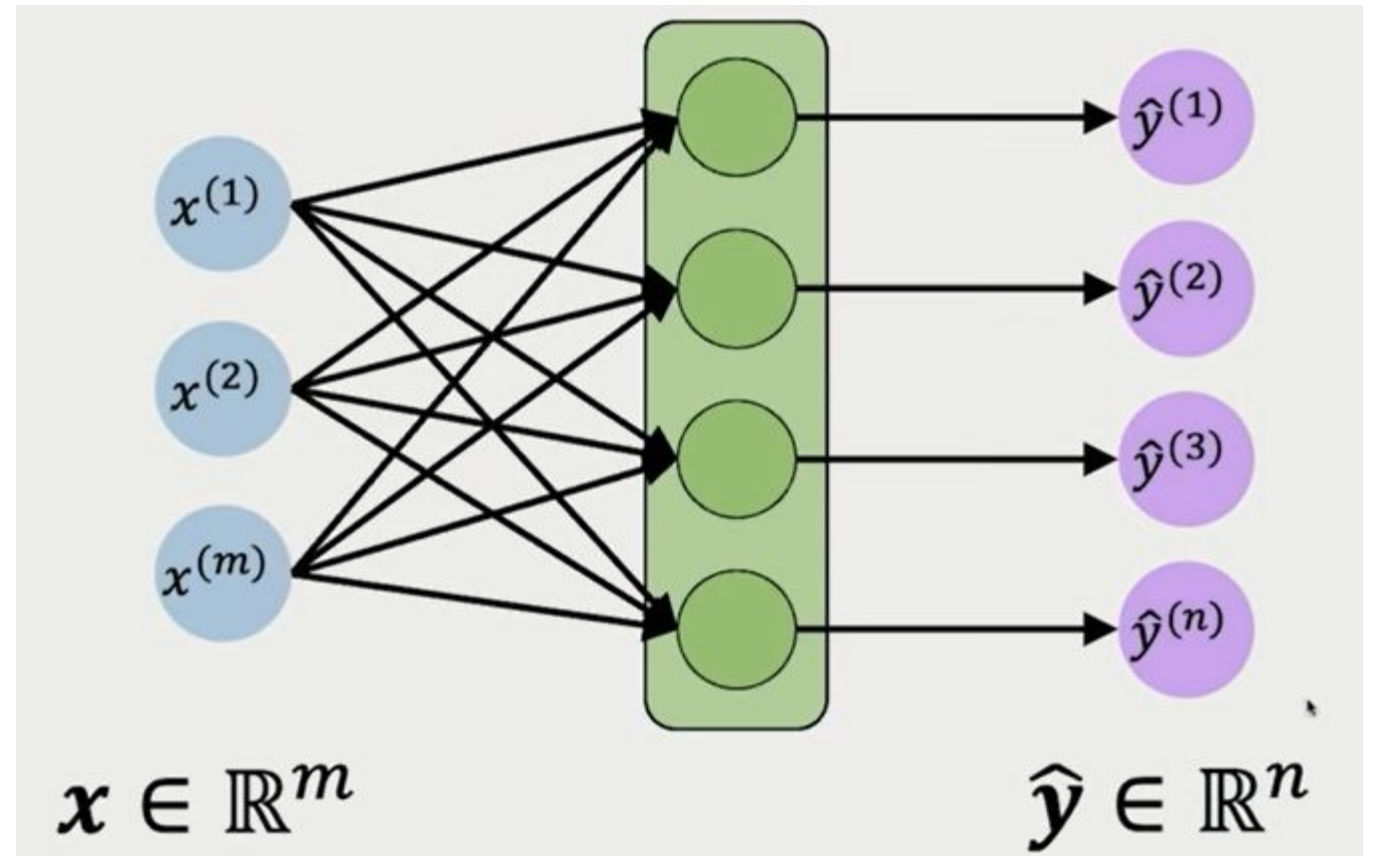
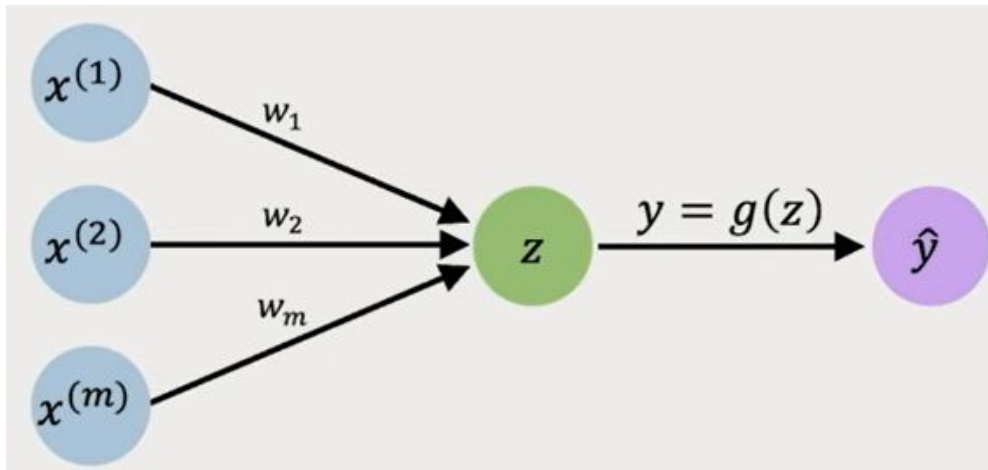
AS SEQUÊNCIAS ESTÃO ENTRE NÓS

Veículos autônomos (predição de trajetórias)

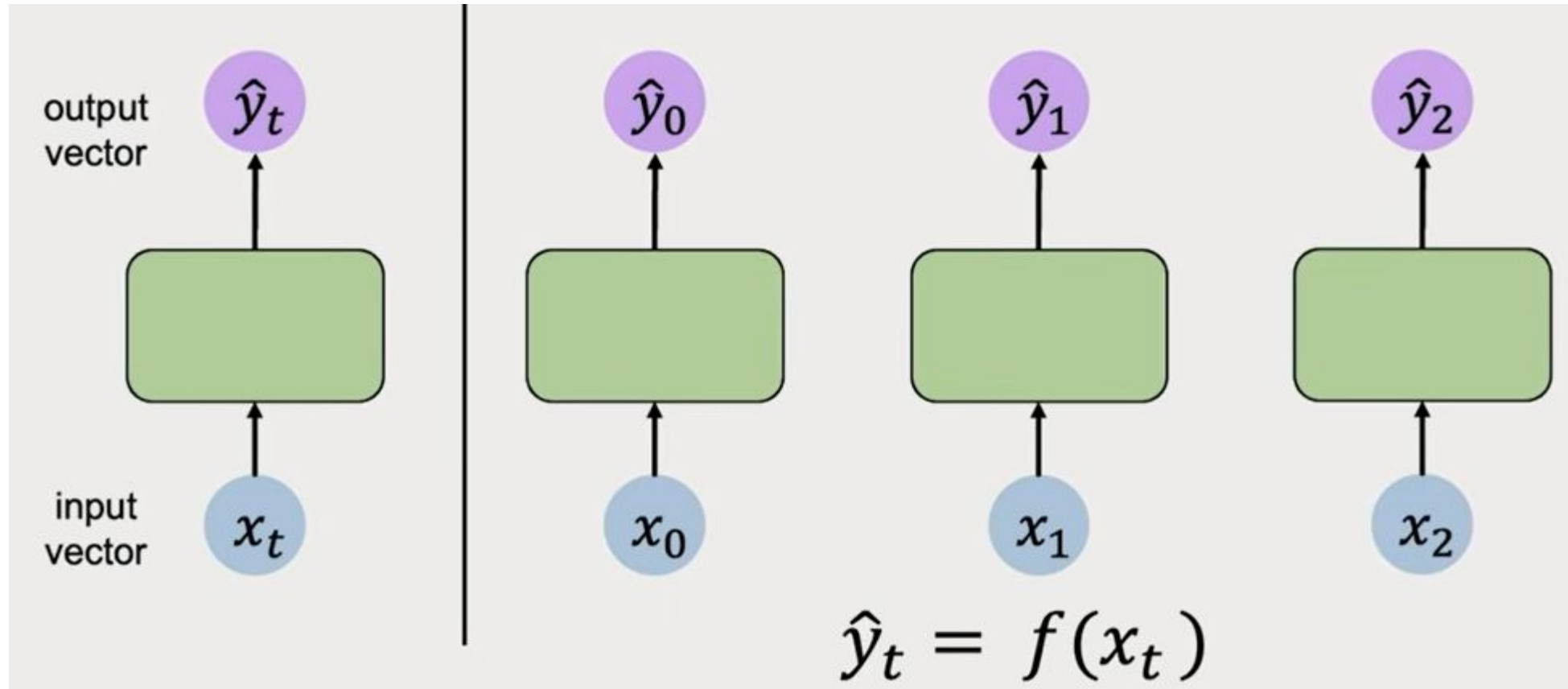


1. **Modelagem Comportamental:** Redes neurais analisam padrões de comportamento humano e de tráfego a partir de vastos conjuntos de dados. Sistema prevê a intenção de outros agentes na estrada (por exemplo, se um pedestre vai atravessar ou não).
2. **Previsão e Planejamento:** Com base na modelagem comportamental, a IA projeta possíveis trajetórias futuras para todos os objetos detectados e, em seguida, planeja uma rota segura e eficiente para o veículo autônomo, evitando colisões e otimizando o fluxo de tráfego.

IDEIAS INICIAIS...RELEMBRANDO FNN

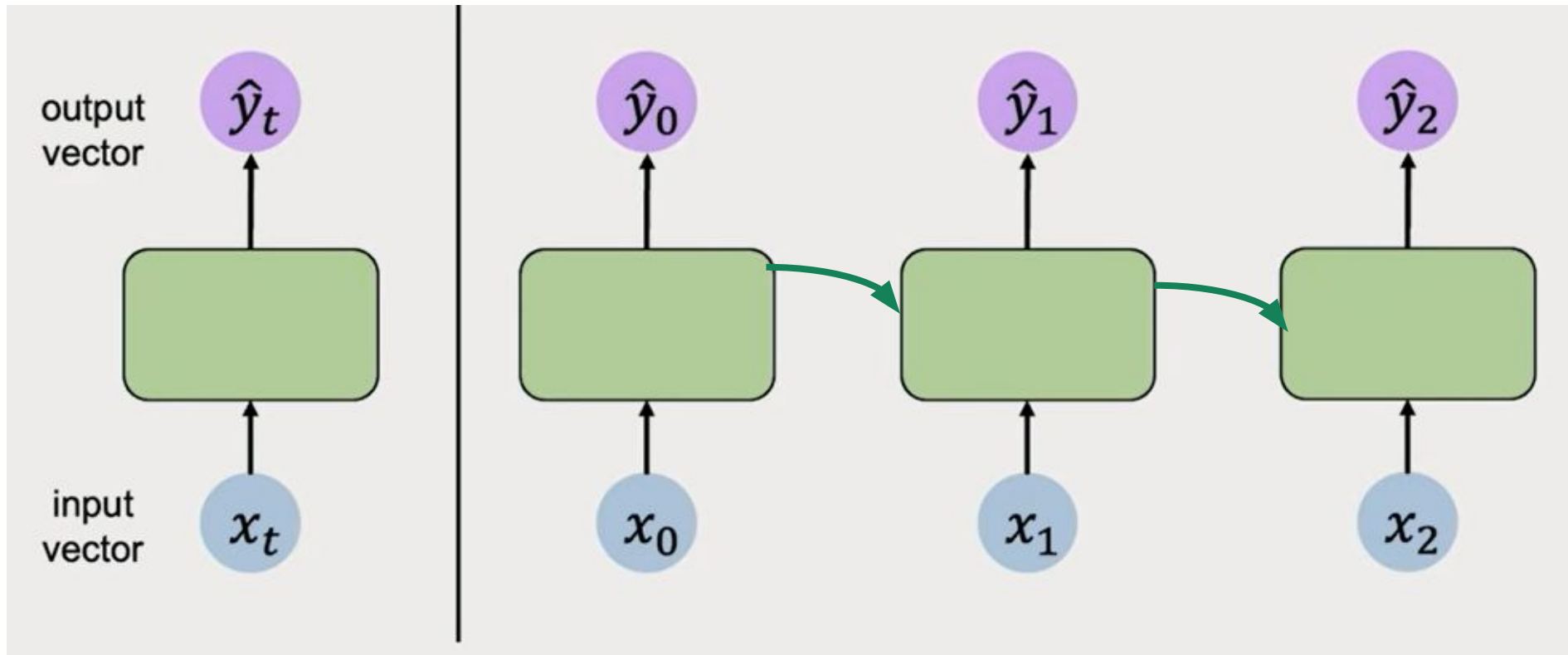


IDEIAS INICIAIS...RNN - CAPTAR DEPENDÊNCIAS



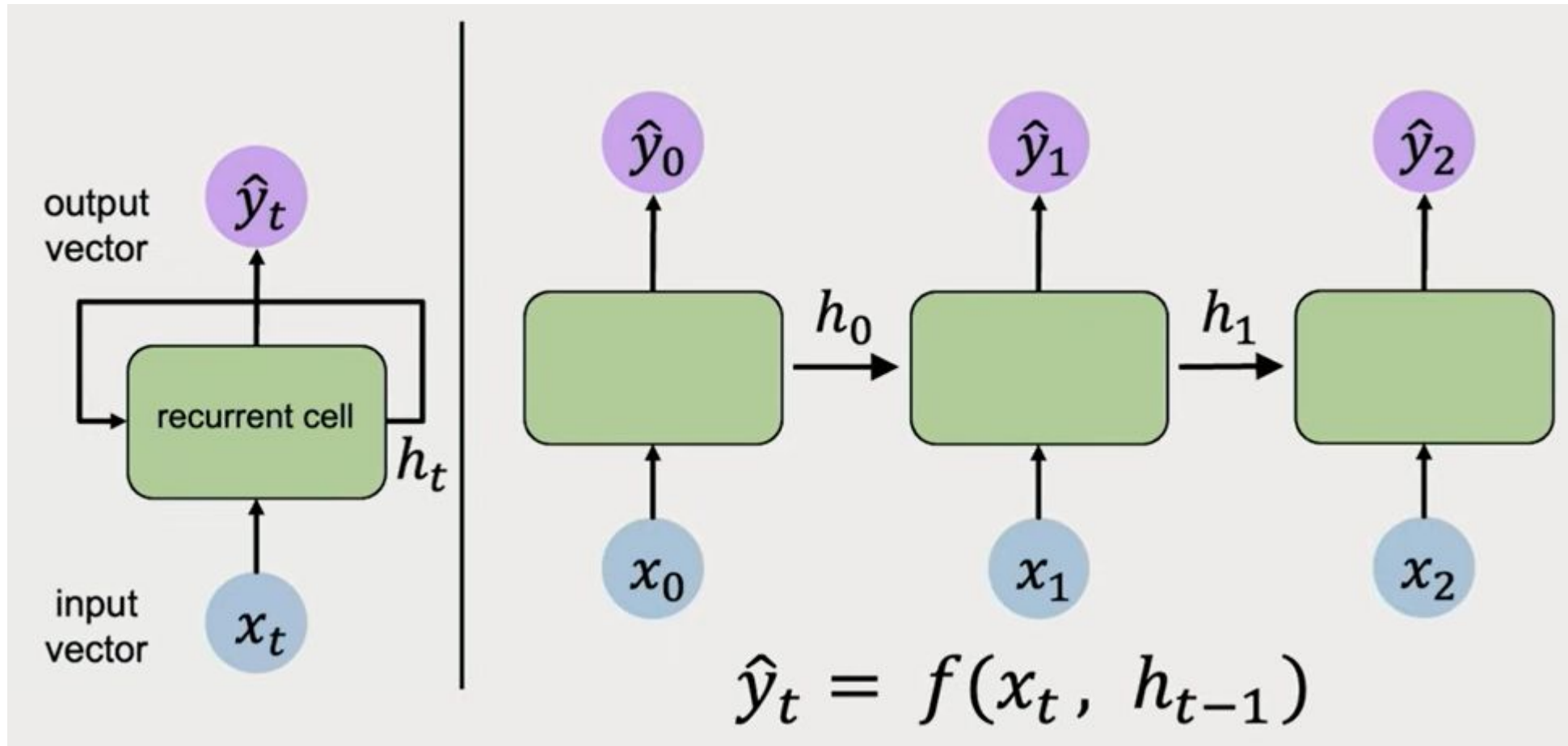
Desenrolar (unroll) a rede - cada intervalo de tempo T com uma célula.
Pode ser visto como camadas ocultas

IDEIAS INICIAIS...RNN - ESTADOS OCULTOS



Introduz-se o conceito de ESTADO OCULTO (*hidden state*), que traz INFO sobre o que a rede aprendeu na entrada anterior. A rede “lembra” de INFO de passos anteriores através dos estados ocultos

ESTADOS OCULTOS...



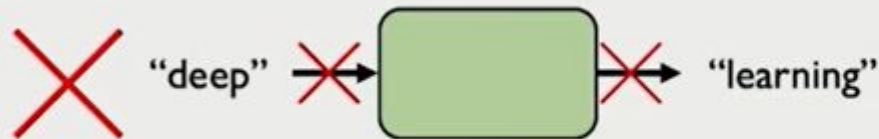
Célula de Memória

BASE - CÉLULA RECORRENTE

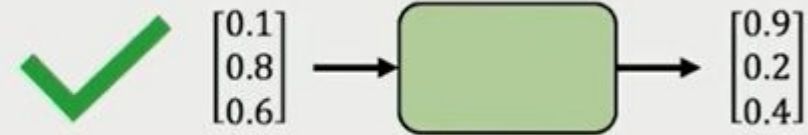
```
hello_world_rnn = RNN()
hidden_state = [0,0,0,0]
sentence = ['RNN', 'é', 'um', 'tipo', 'de', 'rede']

for word in sentence:
    prediction, hidden_state = hello_world_rnn(word,hidden_state)

next_word_prediction = prediction
```



Neural networks cannot interpret words

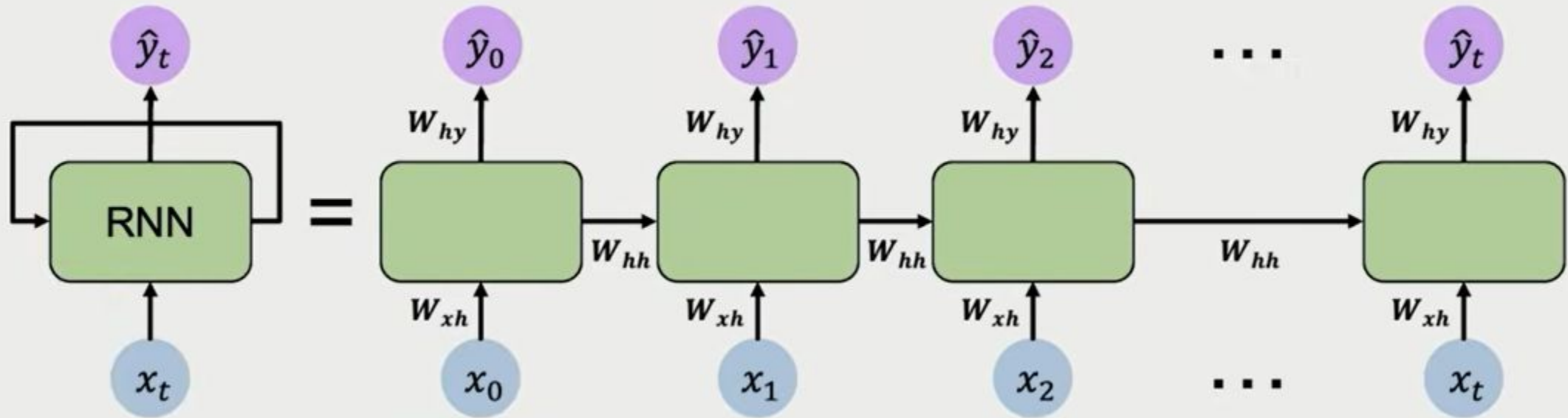


Neural networks require numerical inputs

Necessário codificar. Ideia básica: **tokenização**/embedding:

https://github.com/josenalde/deeplearning/blob/main/src/basic_tokenizer.ipynb

PESOS



Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state function with weights W input old state

tanh: (Vu Pham et al., [2013](#))
ReLU: (Quoc V. Le et al., [2015](#))

Output Vector

$$\hat{y}_t = W_{hy}^T h_t$$

Update Hidden State

$$h_t = \tanh(W_{hh}^T h_{t-1} + W_{xh}^T x_t) + b$$

Input Vector

x_t

PESOS - PSEUDO-CÓDIGO

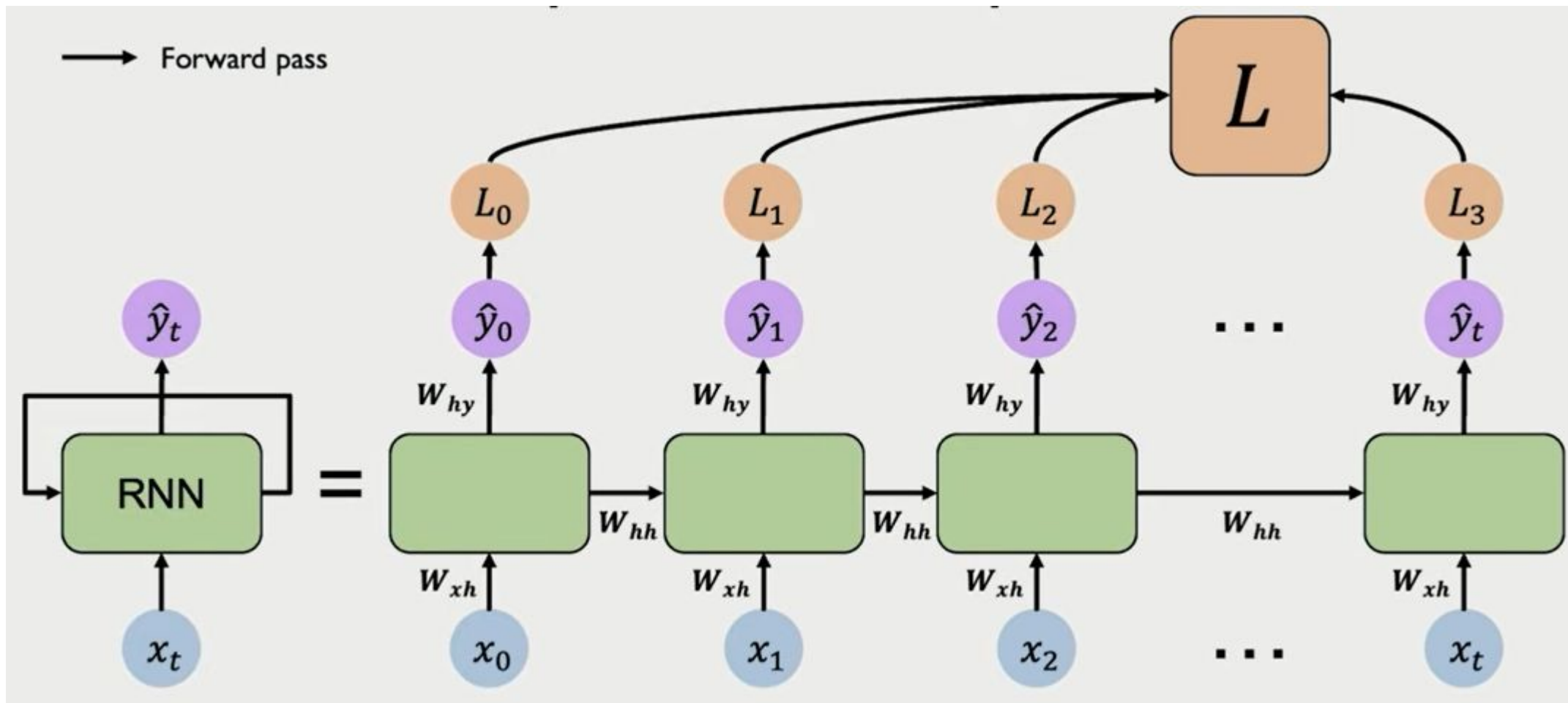
```
class RNNCell(tf.keras.layers.Layer):
    def __init__(self, rnn_units, input_dim, output_dim):
        super(RNNCell, self).__init__()

        self.W_xh = self.add_weight([rnn_units, input_dim])
        self.W_hh = self.add_weight([rnn_units, rnn_units])
        self.W_hy = self.add_weight([output_dim, rnn_units])

        self.h = tf.zeros([rnn_units, 1])

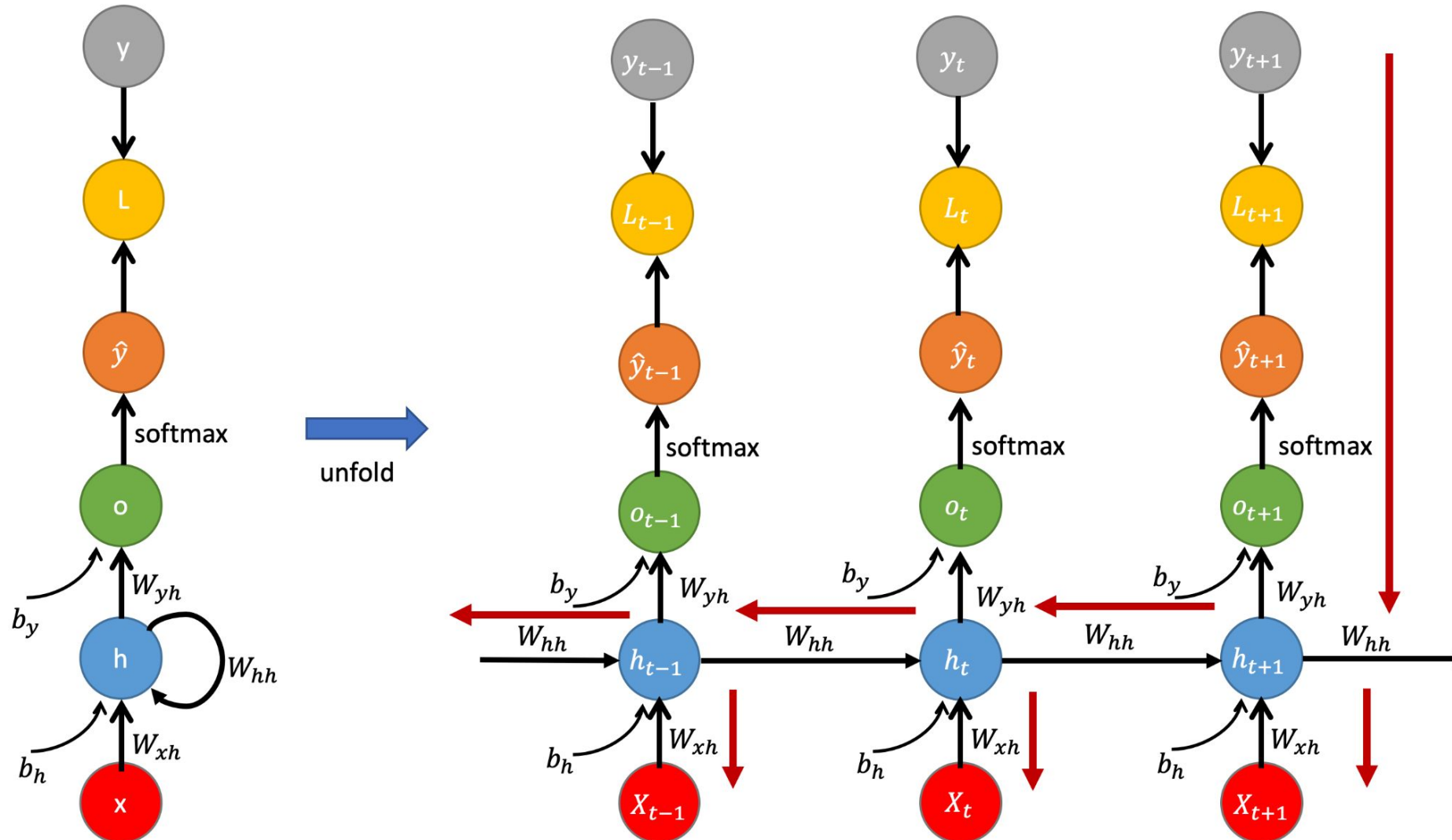
    def call(self, X)
        self.h = tf.math.tanh(self.W_hh * self.h + W_xh * X)
        output = self.W_hy * self.h
```

Treinamento: backpropagation through time ([BPTT](#))

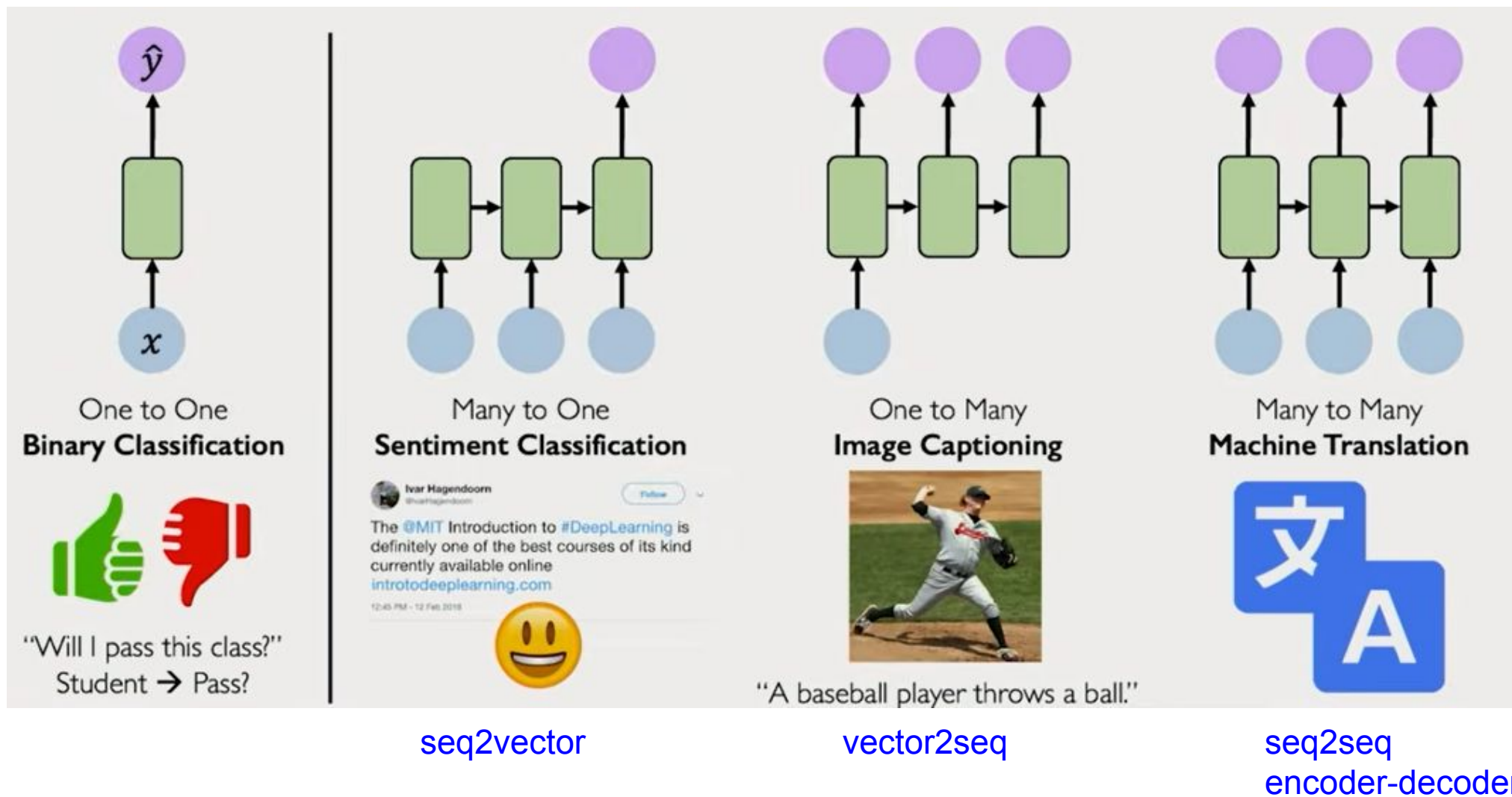


A cada intervalo de tempo, os pesos entre as camadas são iguais - compartilham pesos

Treinamento: backpropagation through time ([BPTT](#))



Combinações de sequências



Combinações de sequências

Exercício proposto (estudo dirigido)

Estudar e implementar as diferentes combinações de sequências com TimeDistributed Layer, seguindo este tutorial:

<https://machinelearningmastery.com/timedistributed-layer-for-long-short-term-memory-networks-in-python/>

Os problemas em lidar com sequências longas

Explosão de gradientes / Desaparecimento de gradientes (gradientes instáveis)

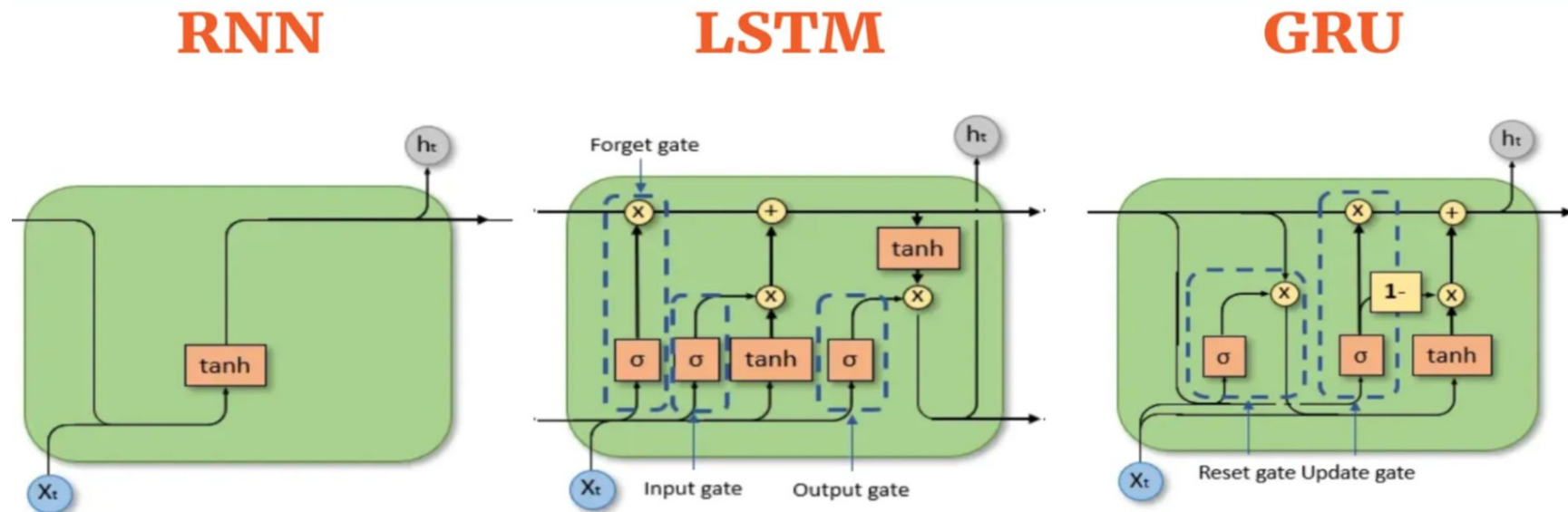
Muitos dos truques com DNN podem ser usados aqui, como boa inicialização de pesos, otimizadores mais rápidos, dropout. Já usar funções não saturantes não “ajuda” (cada intervalo de tempo usa mesmos pesos) - isso explica por que tanh é o padrão

Já BatchNormalization não se usa entre os intervalos de tempo, mas apenas entre as camadas recorrentes. Embora possa usar, a literatura mostra pouco efeito. Há algum resultado com a **LayerNormalization** antes da ativação, que normaliza na dimensão das features e não do batch

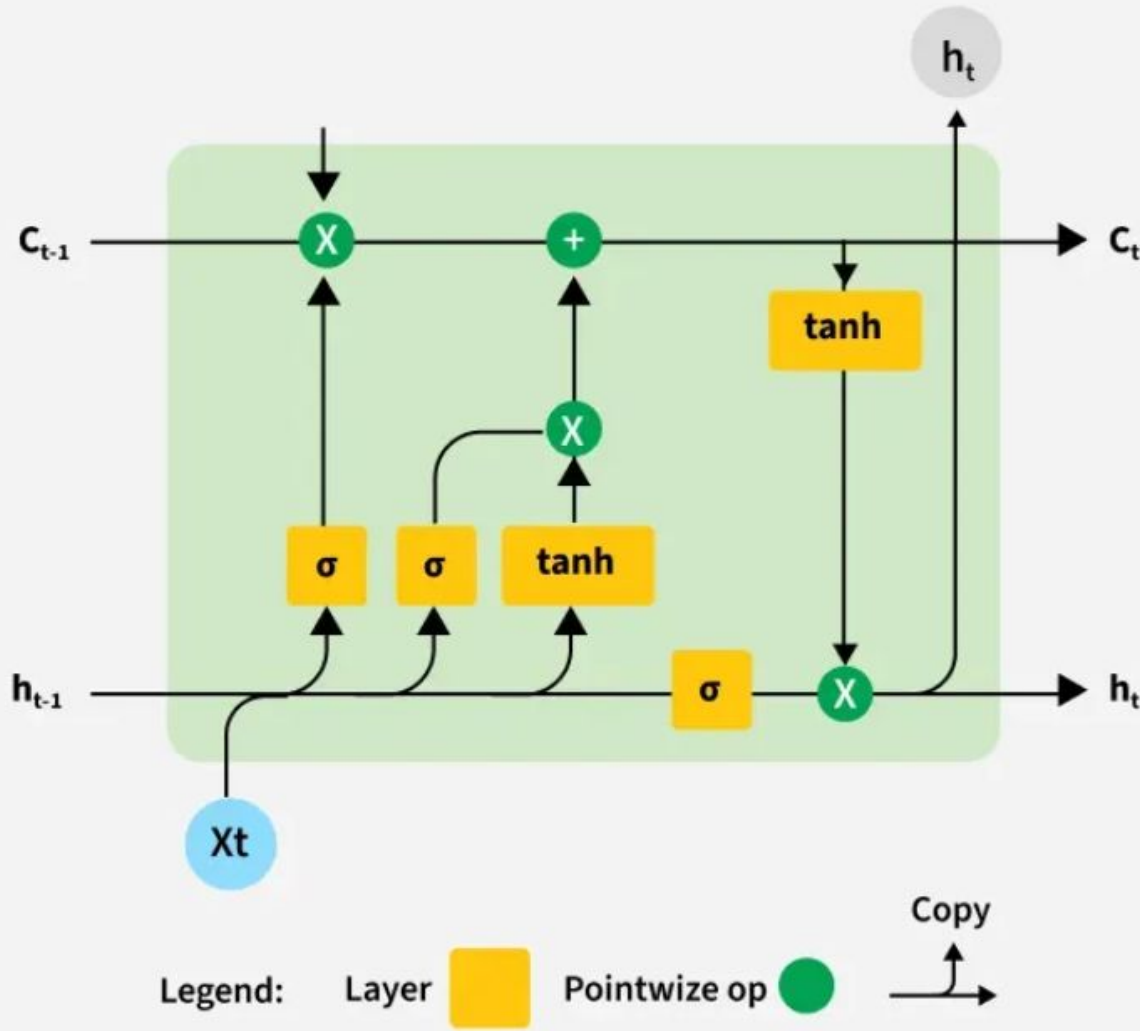
Já o dropout, todas as camadas recorrentes tem o hiperparâmetro **dropout** (taxa aplicada às entradas a cada intervalo de tempo) e **recurrent_dropout** (taxa aplicada para os estados ocultos a cada intervalo de tempo)

Os problemas em lidar com sequências longas

- os dados passam por transformações ao percorrer uma RNN
- algumas informações são perdidas a cada intervalo de tempo
- com o passar do tempo, o estado da RNN praticamente não contém vestígios das primeiras entradas
- Soluções: células com memórias de longo prazo
- Exemplos: Long Short Term Memory (LSTM, 1997) (memória longa de curto prazo)
- Gated Recurrent Unit (GRU, 2014)

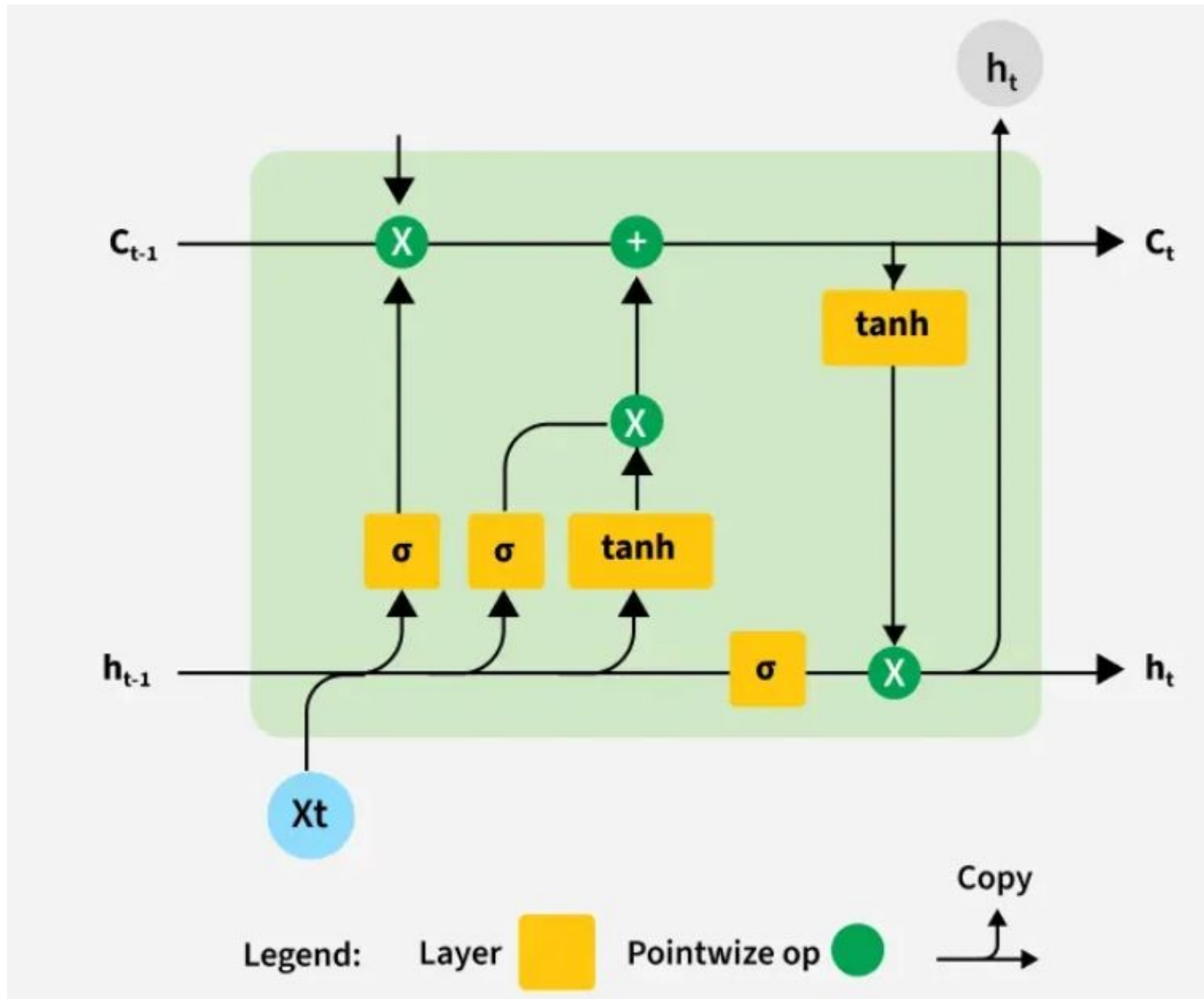


LSTM

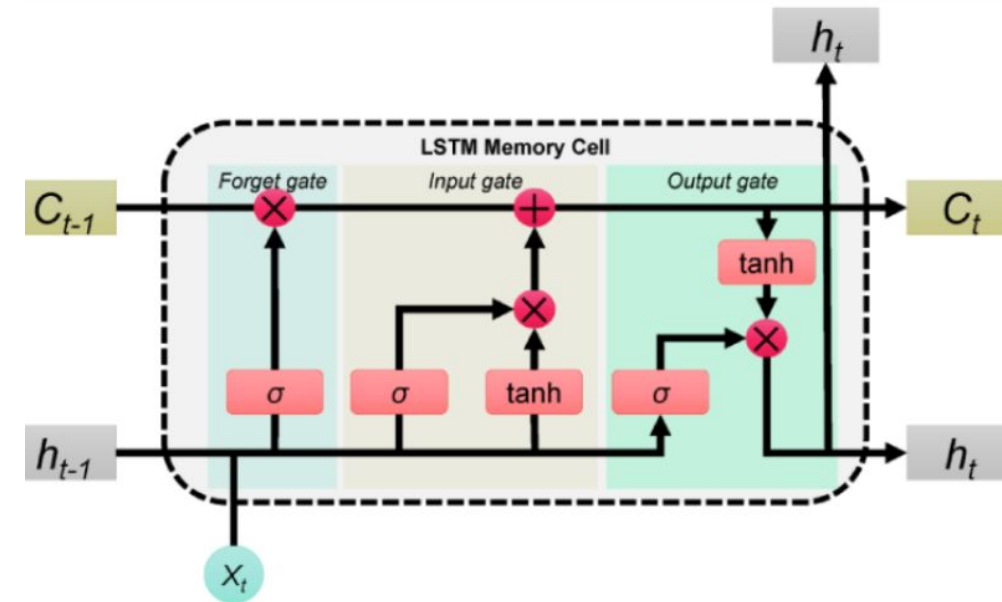


```
model = keras.models.Sequential([  
    keras.layers.LSTM(20, return_sequences=True,  
        input_shape=[None, 1]),  
    keras.layers.LSTM(20, return_sequences=True),  
    keras.layers.TimeDistributed(keras.layers.Dense(10))  
])
```

LSTM

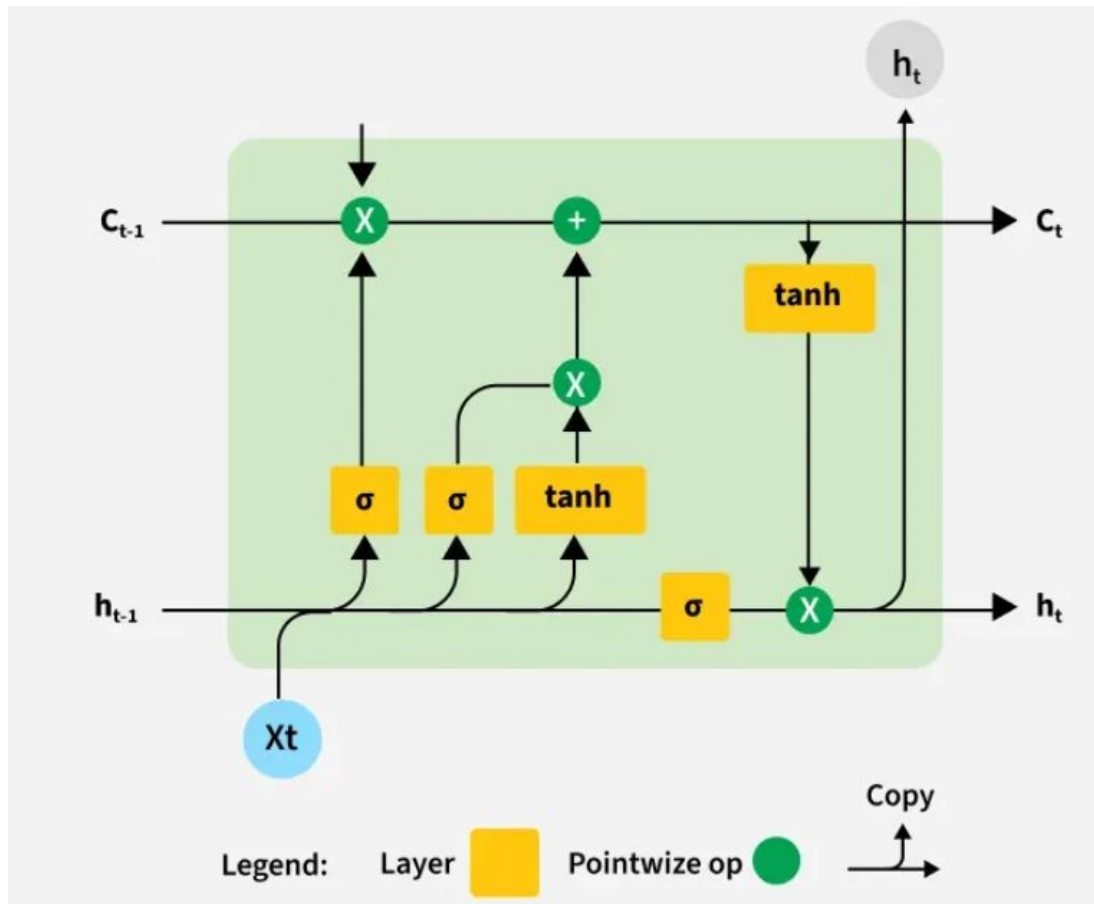


Lembrar o que é importante. Esquecer o que não é mais aplicável. **Como controlar isto?**



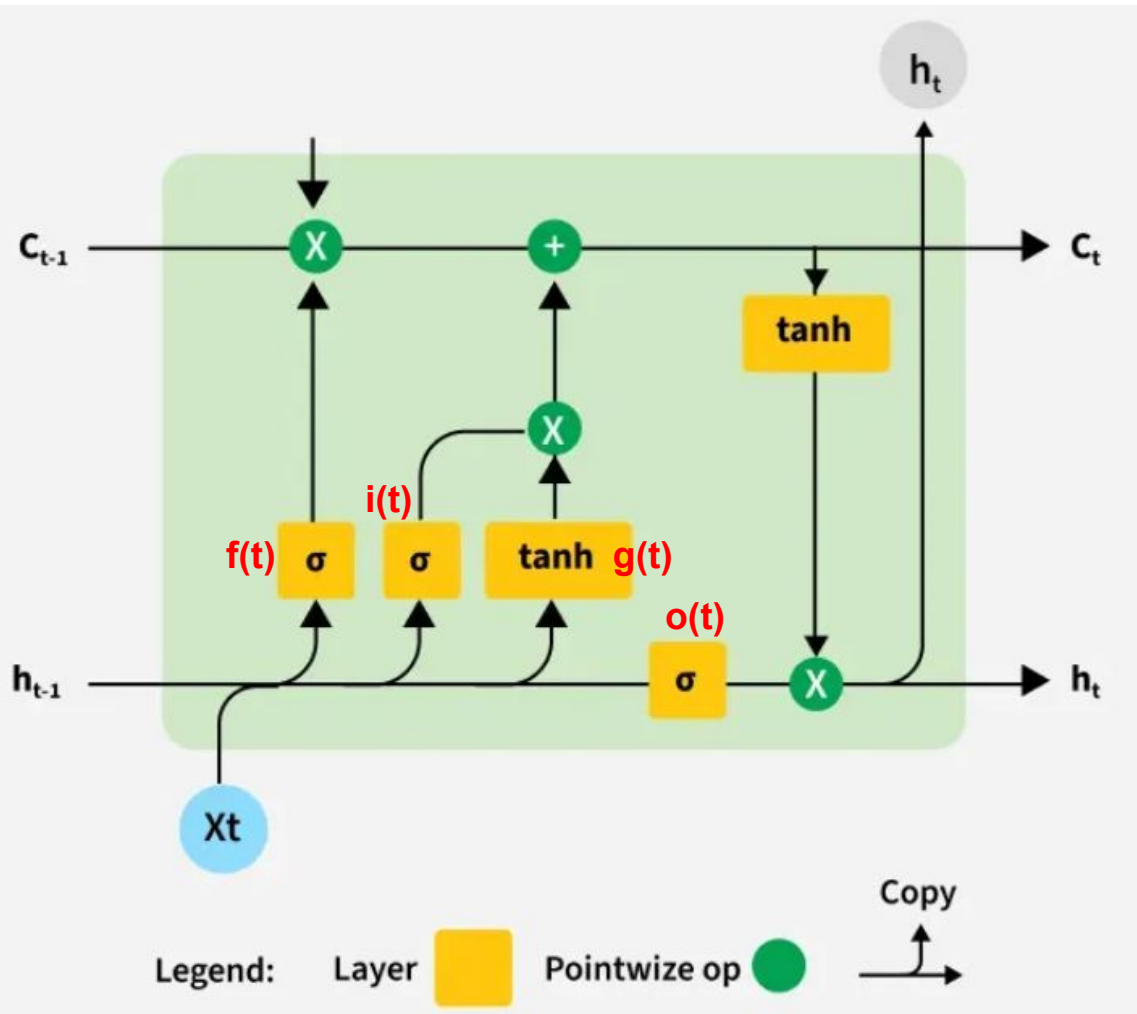
Pense no vetor C como o estado de longo prazo e o vetor h como o de curto prazo

LSTM



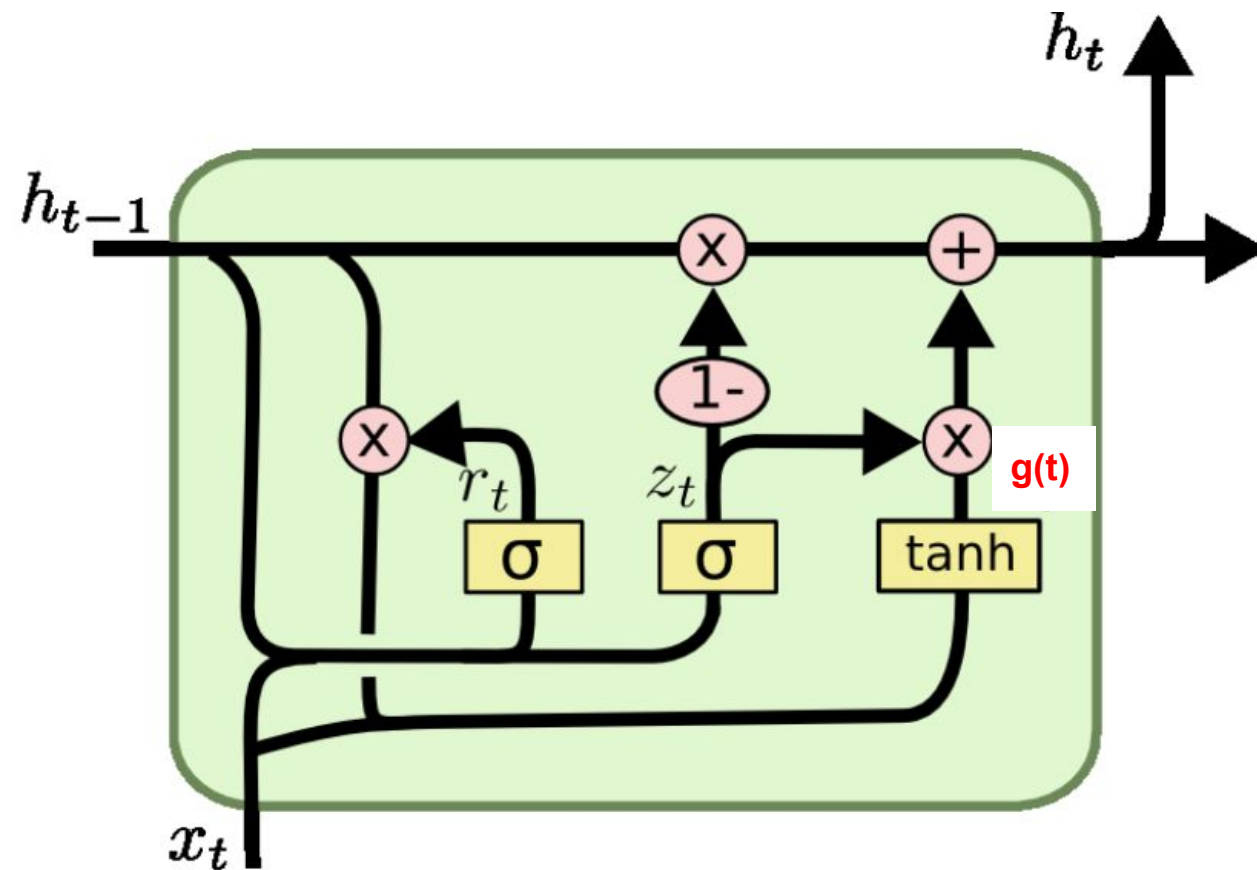
- o estado de longo prazo $c(t-1)$ passa por uma porta esquecida (forget gate) que dropa algumas memórias - *irrelevantes*
- incorpora algumas novas memórias (+), selecionadas pelo portão de entrada (*input gate*)
- O resultado $c(t)$ é copiado para a saída e também passa por **tanh**. O estado é filtrado pelo portão de saída (o que pode sair?), gerando o estado de curto prazo $h(t)$, a saída da célula

LSTM



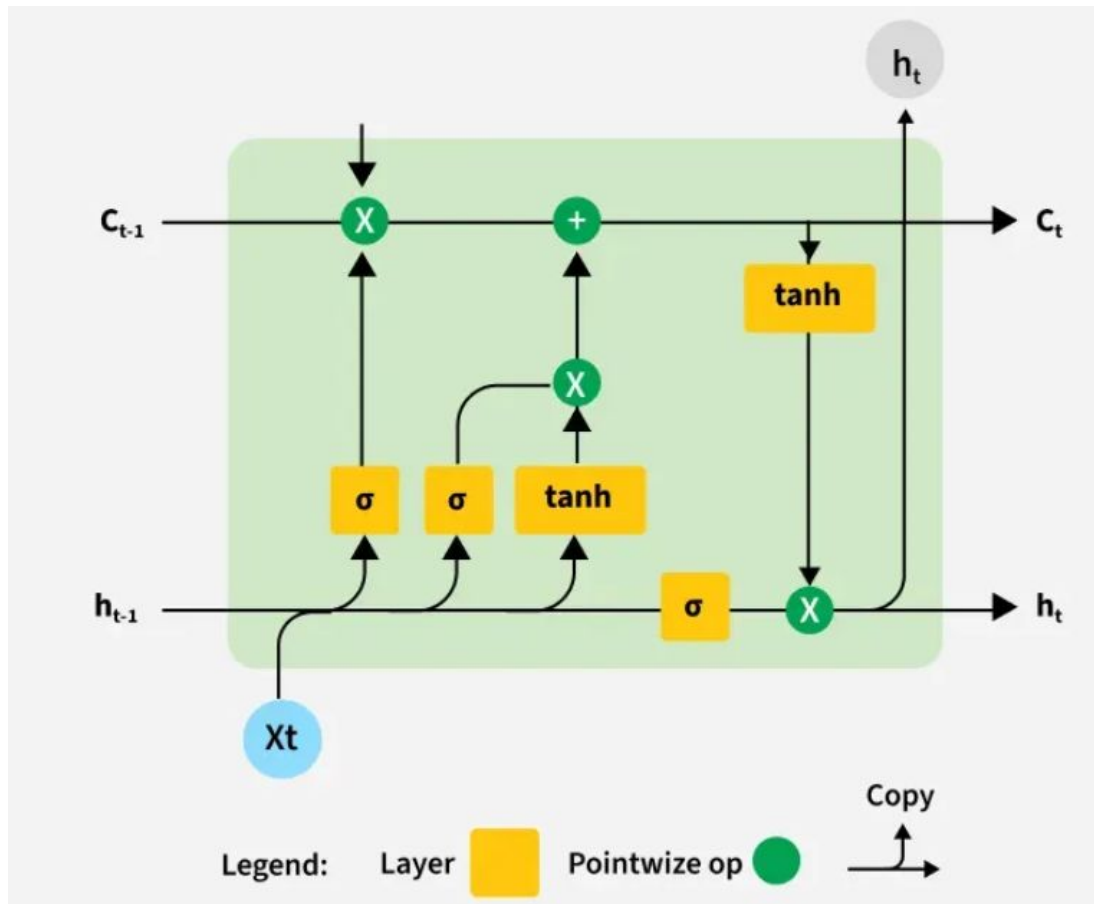
- o vetor $x(t)$ e $h(t-1)$ são fornecidos às quatro camadas densas $f(t)$, $g(t)$, $i(t)$, $o(t)$
- $g(t)$ analisa as entradas atuais e os estados anteriores de curto prazo $h(t-1)$. Suas saídas mais importantes são armazenadas no estado de longo prazo e o resto é dropado
- As camadas $f(t)$, $i(t)$ e $o(t)$ são *gate controllers*. Se gerarem 1 como saída, abrem
- $f(t)$ quais partes de $c(t-1)$ foram deletados
- $i(t)$ controla quais partes do $g(t)$ devem ser adicionadas ao $c(t-1)$
- $o(t)$ controla quais partes de $c(t-1)$ devem ser lidas e geradas neste intervalo de tempo, para $h(t)$ e $y(t)$

GRU



- variante mais simples que LSTM
- ambos os vetores de estado são incorporados com um único vetor $h(t)$
- $z(t)$ controla o forget gate e o input gate
- se $z(t)=1$, forget abre ($=1$) e o input ($1-1=0$). Se $z(t)=0$, é o oposto. Sempre uma memória deve ser armazenada, o local onde ela será armazenada é o primeiro a ser deletado
- não há output gate, o vetor de estado completo é gerado a cada intervalo de tempo
- a diferença é um reset gate ($r(t)$) que controla qual parte do estado anterior é exibida para a camada principal $g(t)$

LSTM



- o estado de longo prazo $c(t-1)$ passa por uma porta esquecida (forget gate) que dropa algumas memórias - *irrelevantes*
- incorpora algumas novas memórias (+), selecionadas pelo portão de entrada (*input gate*)
- O resultado $c(t)$ é copiado para a saída e também passa por **tanh**. O estado é filtrado pelo portão de saída (o que pode sair?), gerando o estado de curto prazo $h(t)$, a saída da célula

LSTM e GRU: não lidam bem com sequências muito longas

```
model = keras.models.Sequential([
    keras.layers.Conv1D(filters=20, kernel_size=4, strides=2, padding='valid',
                        input_shape=[None,1]), #stride!=1 encurta sequencias
    keras.layers.GRU(20, return_sequences=True),
    keras.layers.GRU(20, return_sequences=True),
    keras.layers.TimeDistributed(keras.layers.Dense(10))
])
```

- mapa de características 1D: cada kernel aprenderá a detectar um único padrão sequencial muito curto. Por exemplo, 10 kernels, a saída será 10 sequências unidimensionais
- o tamanho do kernel maior que o stride faz com que todas as entradas sejam usadas para calcular a saída da camada e assim o modelo pode aprender a preservar as informações úteis, descartando apenas os detalhes sem importância

Pesquisar sobre [WaveNet](#) (sem RNN, apenas 1Ds)

Exercícios

Pesquisar sobre [WaveNet](#) (sem RNN, apenas 1Ds) e sua implementação para processamento de áudio

Executar e estudar o exemplo de geração de música no estilo de Bach, de acordo com o tutorial em Bach Corales em

https://github.com/ageron/handson-ml2/blob/master/15_processing_sequences_using_rnns_and_cnns.ipynb